WCAG

- **WCAG POUR Principles**

POUR stands for the four foundational principles of web accessibility. All WCAG guidelines fall under these principles.

1. Perceivable

- **What:** Information must be presented in ways users can perceive (see, hear, or otherwise sense).
- **Key Rules:**
  - Provide text alternatives for non-text content (alt text)
  - Ensure sufficient color contrast
  - Don't use color alone to convey information
  - Make content adaptable (responsive, zoomable)

2. Operable

- **What:** Interface components must be operable by all users.
- **Key Rules:**
  - Full keyboard accessibility
  - No keyboard traps
  - Enough time to read/use content
  - No content that causes seizures
  - Clear navigation

3. Understandable

- **What:** Information and operation must be understandable.
- **Key Rules:**
  - Readable, predictable content
  - Clear labels and instructions
  - Help users avoid/correct errors
  - Consistent navigation

4. Robust

- **What:** Content must be robust enough to work with current and future tools.
- **Key Rules:**
  - Use valid, semantic HTML
  - Ensure compatibility with assistive technologies
  - Follow web standards

- ✓ **Remember Key Success Criteria:**
  - Color alone cannot convey information (affects color-blind users)
  - Keyboard navigation must be fully functional (affects motor-impaired users)
  - Color contrast ratios: 4.5:1 for normal text, 3:1 for large text
  - Semantic HTML is important for screen reader users
  - No keyboard traps (Level A requirement)
  - Captions for pre-recorded audio (Level A)
  - Backwards compatibility means meeting newer standards doesn't invalidate previous compliance
- ✓ **Common Pitfalls:** Don't confuse "what's nice to have" with "what's required by WCAG." Level AAA is recommended but not required; Level AA is typically the target for most websites.

XML

- **XML** (*Extensible Markup Language*) is a flexible markup language and file format used to structure, store, and transport data in a human-readable and machine-readable way, allowing users to define custom tags for describing data.
- **Elements vs. Attributes Strategy:**
  - **Use Elements for:** Actual data content, multiple values, hierarchical data, future extensibility
  - **Use Attributes for:** Metadata about the element, single values, IDs, references
  - **Critical Rule:** Attributes cannot contain multiple values or complex data
- **XML Syntax Fundamentals:**
  - Entity references (<, >, &) escape special characters
  - XML is extensible by design - new elements should not break old parsers

- Case sensitivity matters: <Book> ≠ <book>
- Well-formedness requires proper nesting and closing tags

- **Design Thinking:** When evaluating XML structures, consider future changes. Elements are generally more flexible than attributes for evolving data models.

XML Core Concepts

About Extensibility vs. Character Encoding:

- When studying XML's *flexibility*, note that there are ***two distinct features*** that developers sometimes confuse:
  1. The ability to **extend** documents with new elements without breaking old parsers
  2. The system for handling **special characters** within text content
- These are separate mechanisms serving different purposes - one for structure evolution, one for text content safety

About Case Sensitivity in Markup:

- Unlike some other languages, XML maintains **strict distinction** between uppercase and lowercase in element names
- This design choice affects **consistency requirements** throughout document structure
- Consider how this differs from HTML's approach to case handling

About Document Structure Rules:

- XML's requirement for a single containing element is **fundamental to its tree structure**
- This contrasts with formats that allow multiple top-level items or fragments
- Understanding why this rule exists helps predict how XML processors work

XML Sample

`breakfast_menu`

Familiarize and understand:

```
<?xml version="1.0" encoding="UTF-8"?>
<breakfast_menu>
<food>
    <name>Belgian Waffles</name>
    <price>$5.95</price>
    <description>
   Two of our famous Belgian Waffles with
plenty of real maple syrup
   </description>
    <calories>650</calories>
</food>
<food>
    <name>Homestyle Breakfast</name>
    <price>$6.95</price>
    <description>
    Two eggs, bacon or sausage, toast, and
our ever-popular hash browns
    </description>
    <calories>950</calories>
</food>
</breakfast_menu>
```

Application Integration

- **Integration Level Identification:**
  - **Presentation-Level:** UI layer integration, screen scraping, no backend access
  - **Data Integration:** Shared databases, data warehousing
  - **Business Process Integration:** Workflow automation between systems
  - **Communications-Level:** APIs, messaging, loose coupling
- **Risk Assessment:** Screen scraping is fragile (breaks with UI changes) but sometimes necessary for legacy systems.
- **Solution Matching:** Match the integration type to the business need:
  - Unified dashboard = Presentation-Level
  - Automated workflows = Business Process
  - System independence = Communications-Level/APIs
  - Legacy system access = Presentation-Level (when no API exists)